Advanced Rust (2026): Systems Interfaces

Lecture 6

Lukáš Hozda

Spring 2026

MFF CUNI

Boundary Programming

Systems Interfaces

- Rust code has type, lifetime, layout, and aliasing rules

Systems Interfaces

- Rust code has type, lifetime, layout, and aliasing rules
- C libraries, kernels, CPUs, and wire formats have different rules

Systems Interfaces

- Rust code has type, lifetime, layout, and aliasing rules
- C libraries, kernels, CPUs, and wire formats have different rules
- Systems Rust is mostly about translating between these rule sets

Systems Interfaces

- Rust code has type, lifetime, layout, and aliasing rules
- C libraries, kernels, CPUs, and wire formats have different rules
- Systems Rust is mostly about translating between these rule sets
- `unsafe` is where we state the part Rust cannot verify

Boundary Examples

- C ABI: functions, structs, strings, callbacks

Boundary Examples

- C ABI: functions, structs, strings, callbacks
- Operating system: file descriptors, signals, sockets, memory mappings

Boundary Examples

- C ABI: functions, structs, strings, callbacks
- Operating system: file descriptors, signals, sockets, memory mappings
- CPU: alignment, atomics, SIMD, inline assembly

Boundary Examples

- C ABI: functions, structs, strings, callbacks
- Operating system: file descriptors, signals, sockets, memory mappings
- CPU: alignment, atomics, SIMD, inline assembly
- Build system: linkers, target triples, native dependencies

One Boundary, Two Contracts

- Rust side: values must be valid for their Rust types

One Boundary, Two Contracts

- Rust side: values must be valid for their Rust types
- Foreign side: bytes and handles must satisfy external preconditions

One Boundary, Two Contracts

- Rust side: values must be valid for their Rust types
- Foreign side: bytes and handles must satisfy external preconditions
- Safe wrapper: checks or encodes the foreign preconditions

One Boundary, Two Contracts

- Rust side: values must be valid for their Rust types
- Foreign side: bytes and handles must satisfy external preconditions
- Safe wrapper: checks or encodes the foreign preconditions
- Unsafe block: the smallest place where the two models meet

Representation and Layout

repr(Rust)

- Default Rust layout is not a stable interface

repr(Rust)

- Default Rust layout is not a stable interface
- Field order, padding, and enum layout are compiler choices

repr(Rust)

- Default Rust layout is not a stable interface
- Field order, padding, and enum layout are compiler choices
- The compiler may exploit invalid values for niche optimization

`repr(Rust)`

- Default Rust layout is not a stable interface
- Field order, padding, and enum layout are compiler choices
- The compiler may exploit invalid values for niche optimization
- Do not expose `repr(Rust)` types across FFI or on-disk formats

repr(C)

- `repr(C)` asks Rust to use the platform C ABI layout

repr(C)

- `repr(C)` asks Rust to use the platform C ABI layout
- It fixes field order and uses C-like padding and alignment

`repr(C)`

- `repr(C)` asks Rust to use the platform C ABI layout
- It fixes field order and uses C-like padding and alignment
- It does not make the fields safe to read as raw bytes

repr(C)

- `repr(C)` asks Rust to use the platform C ABI layout
- It fixes field order and uses C-like padding and alignment
- It does not make the fields safe to read as raw bytes
- It does not define endianness for external protocols

Layout Inspection

```
use std::mem::{align_of, size_of};

#[repr(C)]
struct Header {
    tag: u8,
    len: u32,
}

println!("size  = {}", size_of::<Header>());
println!("align = {}", align_of::<Header>());
```

Layout Inspection

```
use std::mem::{align_of, size_of};

#[repr(C)]
struct Header {
    tag: u8,
    len: u32,
}

println!("size  = {}", size_of::<Header>());
println!("align = {}", align_of::<Header>());
```

The likely size is 8: one byte of data, three bytes of padding, four bytes of data

Padding Is Not Data

```rust
#[repr(C)]
#[derive(Clone, Copy)]
struct Header {
    tag: u8,
    len: u32,
}

fn bytes(h: Header) -> [u8; 8] {
    unsafe { std::mem::transmute(h) }
}
```

Padding Is Not Data

```rust
#[repr(C)]
#[derive(Clone, Copy)]
struct Header {
    tag: u8,
    len: u32,
}

fn bytes(h: Header) -> [u8; 8] {
    unsafe { std::mem::transmute(h) }
}
```

- This can read uninitialized padding bytes

Padding Is Not Data

```rust
#[repr(C)]
#[derive(Clone, Copy)]
struct Header {
    tag: u8,
    len: u32,
}

fn bytes(h: Header) -> [u8; 8] {
    unsafe { std::mem::transmute(h) }
}
```

- This can read uninitialized padding bytes
- Miri can report this even when ordinary tests appear fine

Padding Is Not Data

```rust
#[repr(C)]
#[derive(Clone, Copy)]
struct Header {
    tag: u8,
    len: u32,
}

fn bytes(h: Header) -> [u8; 8] {
    unsafe { std::mem::transmute(h) }
}
```

- This can read uninitialized padding bytes
- Miri can report this even when ordinary tests appear fine
- Serialize fields explicitly instead

Field-by-Field Encoding

```rust
fn encode_header(tag: u8, len: u32) -> [u8; 5] {
    let mut out = [0_u8; 5];
    out[0] = tag;
    out[1..5].copy_from_slice(&len.to_le_bytes());
    out
}
```

Field-by-Field Encoding

```rust
fn encode_header(tag: u8, len: u32) -> [u8; 5] {
    let mut out = [0_u8; 5];
    out[0] = tag;
    out[1..5].copy_from_slice(&len.to_le_bytes());
    out
}
```

- The wire format is five bytes here, not `size_of::<Header>()`

Field-by-Field Encoding

```rust
fn encode_header(tag: u8, len: u32) -> [u8; 5] {
    let mut out = [0_u8; 5];
    out[0] = tag;
    out[1..5].copy_from_slice(&len.to_le_bytes());
    out
}
```

- The wire format is five bytes here, not `size_of::<Header>()`
- Endianness is chosen by the protocol

Field-by-Field Encoding

```rust
fn encode_header(tag: u8, len: u32) -> [u8; 5] {
    let mut out = [0_u8; 5];
    out[0] = tag;
    out[1..5].copy_from_slice(&len.to_le_bytes());
    out
}
```

- The wire format is five bytes here, not `size_of::<Header>()`
- Endianness is chosen by the protocol
- Padding never enters the representation

Rust Values Have Validity Rules

- `bool` may only contain `0` or `1`

Rust Values Have Validity Rules

- `bool` may only contain `0` or `1`
- `char` must be a Unicode scalar value

Rust Values Have Validity Rules

- `bool` may only contain `0` or `1`
- `char` must be a Unicode scalar value
- References must be non-null, aligned, valid, and obey aliasing rules

Rust Values Have Validity Rules

- `bool` may only contain `0` or `1`
- `char` must be a Unicode scalar value
- References must be non-null, aligned, valid, and obey aliasing rules
- Creating an invalid value is already undefined behavior

C Booleans and Rust Booleans

```
#[repr(C)]
struct CRecord {
    tag: libc::c_int,
    enabled: libc::c_int,
}

fn enabled(record: CRecord) -> bool {
    record.enabled != 0
}
```

C Booleans and Rust Booleans

```
#[repr(C)]
struct CRecord {
    tag: libc::c_int,
    enabled: libc::c_int,
}

fn enabled(record: CRecord) -> bool {
    record.enabled != 0
}
```

A C API that says “boolean” often means integer convention, not Rust `bool`

C Enums

```c
enum Mode {
    MODE_READ = 1,
    MODE_WRITE = 2,
};
```

C Enums

```
enum Mode {
    MODE_READ = 1,
    MODE_WRITE = 2,
};
```

```
#[repr(i32)]
enum Mode {
    Read = 1,
    Write = 2,
}
```

C Enums

```
enum Mode {
    MODE_READ = 1,
    MODE_WRITE = 2,
};
```

```
#[repr(i32)]
enum Mode {
    Read = 1,
    Write = 2,
}
```

- A C caller can still pass 17

C Enums

```
enum Mode {
    MODE_READ = 1,
    MODE_WRITE = 2,
};
```

```
#[repr(i32)]
enum Mode {
    Read = 1,
    Write = 2,
}
```

- A C caller can still pass `17`
- Rust enum values must match one of the variants

C Enums

```
enum Mode {
    MODE_READ = 1,
    MODE_WRITE = 2,
};
```

```
#[repr(i32)]
enum Mode {
    Read = 1,
    Write = 2,
}
```

- A C caller can still pass `17`
- Rust enum values must match one of the variants
- Parse into `Result<Mode, i32>` at the boundary

Packed Field Reference

```
#[repr(C, packed)]
struct PacketHeader {
    tag: u8,
    len: u16,
}

fn len_ref(h: &PacketHeader) -> &u16 {
    &h.len
}
```

Packed Field Reference

```rust
#[repr(C, packed)]
struct PacketHeader {
    tag: u8,
    len: u16,
}

fn len_ref(h: &PacketHeader) -> &u16 {
    &h.len
}
```

Taking a normal reference to `len` would require alignment that the packed field may not have

Packed Field Access

```
fn len_ok(h: &PacketHeader) -> u16 {
    let ptr = std::ptr::addr_of!(h.len);
    unsafe { u16::from_le(std::ptr::read_unaligned(ptr)) }
}
```

Packed Field Access

```
fn len_ok(h: &PacketHeader) -> u16 {
    let ptr = std::ptr::addr_of!(h.len);
    unsafe { u16::from_le(std::ptr::read_unaligned(ptr)) }
}
```

- `addr_of!` avoids creating an intermediate reference

Packed Field Access

```rust
fn len_ok(h: &PacketHeader) -> u16 {
    let ptr = std::ptr::addr_of!(h.len);
    unsafe { u16::from_le(std::ptr::read_unaligned(ptr)) }
}
```

- `addr_of!` avoids creating an intermediate reference
- `read_unaligned` states the actual load contract

Packed Field Access

```rust
fn len_ok(h: &PacketHeader) -> u16 {
    let ptr = std::ptr::addr_of!(h.len);
    unsafe { u16::from_le(std::ptr::read_unaligned(ptr)) }
}
```

- `addr_of!` avoids creating an intermediate reference
- `read_unaligned` states the actual load contract
- Endianness conversion is still a separate operation

Typed Raw Memory

PhantomData

- Raw pointers do not express ownership or borrowing by themselves

PhantomData

- Raw pointers do not express ownership or borrowing by themselves
- `PhantomData` adds compile-time type and lifetime relationships

PhantomData

- Raw pointers do not express ownership or borrowing by themselves
- `PhantomData` adds compile-time type and lifetime relationships
- It has no runtime storage

PhantomData

- Raw pointers do not express ownership or borrowing by themselves
- `PhantomData` adds compile-time type and lifetime relationships
- It has no runtime storage
- It changes how the compiler reasons about the wrapper

Borrowed Slice Wrapper

```
use std::marker::PhantomData;

struct RawSlice<'a, T> {
    ptr: *const T,
    len: usize,
    _marker: PhantomData<&'a T>,
}
```

Borrowed Slice Wrapper

```rust
use std::marker::PhantomData;

struct RawSlice<'a, T> {
    ptr: *const T,
    len: usize,
    _marker: PhantomData<&'a T>,
}
```

The marker says: this value behaves as if it borrowed `T` values for `'a`

Constructing the Wrapper

```
impl<'a, T> RawSlice<'a, T> {
    unsafe fn from_raw_parts(ptr: *const T, len: usize) -> Self {
        Self {
            ptr,
            len,
            _marker: PhantomData,
        }
    }
}
```

Constructing the Wrapper

```
impl<'a, T> RawSlice<'a, T> {
    unsafe fn from_raw_parts(ptr: *const T, len: usize) -> Self {
        Self {
            ptr,
            len,
            _marker: PhantomData,
        }
    }
}
```

The unsafe constructor must require the same preconditions as `slice::from_raw_parts`

Exposing the Slice

```
impl<'a, T> RawSlice<'a, T> {
    fn as_slice(&self) -> &'a [T] {
        unsafe { std::slice::from_raw_parts(self.ptr, self.len) }
    }
}
```

Exposing the Slice

```
impl<'a, T> RawSlice<'a, T> {
    fn as_slice(&self) -> &'a [T] {
        unsafe { std::slice::from_raw_parts(self.ptr, self.len) }
    }
}
```

- The safe method is only sound if the constructor contract was true

Exposing the Slice

```
impl<'a, T> RawSlice<'a, T> {
    fn as_slice(&self) -> &'a [T] {
        unsafe { std::slice::from_raw_parts(self.ptr, self.len) }
    }
}
```

- The safe method is only sound if the constructor contract was true
- This is a common shape: unsafe construction, safe observation

Marker Shapes

- `PhantomData<T>`: wrapper logically owns values of type `T`

Marker Shapes

- `PhantomData<T>`: wrapper logically owns values of type `T`
- `PhantomData<&'a T>`: wrapper logically shares values for `'a`

Marker Shapes

- `PhantomData<T>`: wrapper logically owns values of type `T`
- `PhantomData<&'a T>`: wrapper logically shares values for `'a`
- `PhantomData<&'a mut T>`: wrapper logically has exclusive access for `'a`

Marker Shapes

- `PhantomData<T>`: wrapper logically owns values of type `T`
- `PhantomData<&'a T>`: wrapper logically shares values for `'a`
- `PhantomData<&'a mut T>`: wrapper logically has exclusive access for `'a`
- `PhantomData<fn() -> T>`: mentions `T` without ownership

Variance and Drop Check

- A lifetime marker can prevent a raw pointer wrapper from outliving borrowed data

Variance and Drop Check

- A lifetime marker can prevent a raw pointer wrapper from outliving borrowed data
- An ownership marker affects drop checking

Variance and Drop Check

- A lifetime marker can prevent a raw pointer wrapper from outliving borrowed data
- An ownership marker affects drop checking
- Marker choice can affect `Send`, `Sync`, and variance

Variance and Drop Check

- A lifetime marker can prevent a raw pointer wrapper from outliving borrowed data
- An ownership marker affects drop checking
- Marker choice can affect `Send`, `Sync`, and variance
- If the marker is wrong, the public type tells a false story

Untyped Allocation

```
struct ForeignArray<T> {
    ptr: std::ptr::NonNull<T>,
    len: usize,
    drop_fn: unsafe extern "C" fn(*mut T, usize),
    _marker: PhantomData<T>,
}
```

Untyped Allocation

```
struct ForeignArray<T> {
    ptr: std::ptr::NonNull<T>,
    len: usize,
    drop_fn: unsafe extern "C" fn(*mut T, usize),
    _marker: PhantomData<T>,
}
```

- The pointer came from somewhere else

Untyped Allocation

```
struct ForeignArray<T> {
    ptr: std::ptr::NonNull<T>,
    len: usize,
    drop_fn: unsafe extern "C" fn(*mut T, usize),
    _marker: PhantomData<T>,
}
```

- The pointer came from somewhere else
- Rust still needs to know whether `T` is logically owned

Untyped Allocation

```
struct ForeignArray<T> {
    ptr: std::ptr::NonNull<T>,
    len: usize,
    drop_fn: unsafe extern "C" fn(*mut T, usize),
    _marker: PhantomData<T>,
}
```

- The pointer came from somewhere else
- Rust still needs to know whether `T` is logically owned
- `Drop` must call the matching foreign destructor exactly once

Destruction Boundary

```
impl<T> Drop for ForeignArray<T> {
    fn drop(&mut self) {
        unsafe {
            (self.drop_fn)(self.ptr.as_ptr(), self.len);
        }
    }
}
```

Destruction Boundary

```
impl<T> Drop for ForeignArray<T> {
    fn drop(&mut self) {
        unsafe {
            (self.drop_fn)(self.ptr.as_ptr(), self.len);
        }
    }
}
```

This is only sound if `drop_fn` expects exactly this pointer, length, allocator, and element type

C Strings and FFI

Declaring C Functions

```
use libc::{c_char, size_t};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> size_t;
}
```

Declaring C Functions

```
use libc::{c_char, size_t};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> size_t;
}
```

- The declaration is a promise about ABI and signature

Declaring C Functions

```
use libc::{c_char, size_t};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> size_t;
}
```

- The declaration is a promise about ABI and signature
- Rust cannot inspect the C implementation

Declaring C Functions

```
use libc::{c_char, size_t};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> size_t;
}
```

- The declaration is a promise about ABI and signature
- Rust cannot inspect the C implementation
- Calling the function is unsafe unless wrapped by checked Rust code

Rust String to C String

```rust
use std::ffi::CString;

fn strlen_checked(s: &str) -> Result<usize, std::ffi::NulError> {
    let c = CString::new(s)?;
    let n = unsafe { strlen(c.as_ptr()) };
    Ok(n as usize)
}
```

Rust String to C String

```rust
use std::ffi::CString;

fn strlen_checked(s: &str) -> Result<usize, std::ffi::NulError> {
    let c = CString::new(s)?;
    let n = unsafe { strlen(c.as_ptr()) };
    Ok(n as usize)
}
```

- `CString::new` rejects interior NUL bytes

Rust String to C String

```rust
use std::ffi::CString;

fn strlen_checked(s: &str) -> Result<usize, std::ffi::NulError> {
    let c = CString::new(s)?;
    let n = unsafe { strlen(c.as_ptr()) };
    Ok(n as usize)
}
```

- `CString::new` rejects interior NUL bytes
- `as_ptr` is valid only while the `CString` is alive

Rust String to C String

```rust
use std::ffi::CString;

fn strlen_checked(s: &str) -> Result<usize, std::ffi::NulError> {
    let c = CString::new(s)?;
    let n = unsafe { strlen(c.as_ptr()) };
    Ok(n as usize)
}
```

- `CString::new` rejects interior NUL bytes
- `as_ptr` is valid only while the `CString` is alive
- The C function must not store the pointer for later use

C String to Rust String

```
use std::ffi::CStr;

unsafe fn borrowed_name(ptr: *const libc::c_char) -> Option<String> {
    if ptr.is_null() {
        return None;
    }
    Some(CStr::from_ptr(ptr).to_string_lossy().into_owned())
}
```

C String to Rust String

```rust
use std::ffi::CStr;

unsafe fn borrowed_name(ptr: *const libc::c_char) -> Option<String> {
    if ptr.is_null() {
        return None;
    }
    Some(CStr::from_ptr(ptr).to_string_lossy().into_owned())
}
```

`CStr::from_ptr` requires a valid NUL-terminated byte sequence reachable from `ptr`

Foreign Ownership

```
unsafe extern "C" {
    fn strdup(s: *const libc::c_char) -> *mut libc::c_char;
    fn free(ptr: *mut libc::c_void);
}
```

Foreign Ownership

```
unsafe extern "C" {
    fn strdup(s: *const libc::c_char) -> *mut libc::c_char;
    fn free(ptr: *mut libc::c_void);
}
```

- Some returned pointers are borrowed

Foreign Ownership

```
unsafe extern "C" {
    fn strdup(s: *const libc::c_char) -> *mut libc::c_char;
    fn free(ptr: *mut libc::c_void);
}
```

- Some returned pointers are borrowed
- Some returned pointers must be released by a specific function

Foreign Ownership

```
unsafe extern "C" {
    fn strdup(s: *const libc::c_char) -> *mut libc::c_char;
    fn free(ptr: *mut libc::c_void);
}
```

- Some returned pointers are borrowed
- Some returned pointers must be released by a specific function
- The Rust type must encode which case applies

Owned C String Wrapper I

```
use std::{ffi::{CStr, CString}, io};

fn duplicate(s: &str) -> io::Result<String> {
    let input = CString::new(s)
        .map_err(|_| io::Error::from(io::ErrorKind::InvalidInput))?;

    let ptr = unsafe { strdup(input.as_ptr()) };
    if ptr.is_null() {
        return Err(io::Error::last_os_error());
    }
```

```
    let out = unsafe {
        CStr::from_ptr(ptr).to_string_lossy().into_owned()
    };
    unsafe {
        free(ptr.cast());
    }
    Ok(out)
}
```

```
    let out = unsafe {
        CStr::from_ptr(ptr).to_string_lossy().into_owned()
    };
    unsafe {
        free(ptr.cast());
    }
    Ok(out)
}
```

The same allocator family that creates the pointer must destroy it

Callback Boundaries

```
type Callback = unsafe extern "C" fn(ctx: *mut libc::c_void, value: i32);

unsafe extern "C" {
    fn register_callback(cb: Callback, ctx: *mut libc::c_void);
}
```

Callback Boundaries

```
type Callback = unsafe extern "C" fn(ctx: *mut libc::c_void, value: i32);

unsafe extern "C" {
    fn register_callback(cb: Callback, ctx: *mut libc::c_void);
}
```

- The callback may be called later, on another thread, or during registration

Callback Boundaries

```
type Callback = unsafe extern "C" fn(ctx: *mut libc::c_void, value: i32);

unsafe extern "C" {
    fn register_callback(cb: Callback, ctx: *mut libc::c_void);
}
```

- The callback may be called later, on another thread, or during registration
- The context pointer must remain valid for the API's actual lifetime

Callback Boundaries

```
type Callback = unsafe extern "C" fn(ctx: *mut libc::c_void, value: i32);

unsafe extern "C" {
    fn register_callback(cb: Callback, ctx: *mut libc::c_void);
}
```

- The callback may be called later, on another thread, or during registration
- The context pointer must remain valid for the API's actual lifetime
- Panics must not unwind into C unless the ABI explicitly permits it

Callback Context

```
struct State {
    sum: i64,
}

unsafe extern "C" fn on_value(ctx: *mut libc::c_void, value: i32) {
    let state = unsafe { &mut *(ctx.cast::<State>()) };
    state.sum += value as i64;
}
```

Callback Context

```
struct State {
    sum: i64,
}

unsafe extern "C" fn on_value(ctx: *mut libc::c_void, value: i32) {
    let state = unsafe { &mut *(ctx.cast::<State>()) };
    state.sum += value as i64;
}
```

This assumes exclusive access to `State` whenever the C library invokes the callback

Building and Linking

Build Scripts

```
[build-dependencies]
cc = "1"

[dependencies]
libc = "0.2"
```

Build Scripts

```toml
[build-dependencies]
cc = "1"

[dependencies]
libc = "0.2"
```

`build.rs` is compiled for the build host and runs before the Rust crate is compiled

Compiling a C Shim

```rust
fn main() {
    println!("cargo:rerun-if-changed=src/shim.c");
    println!("cargo:rerun-if-changed=src/shim.h");

    cc::Build::new()
        .file("src/shim.c")
        .warnings(true)
        .compile("shim");
}
```

Compiling a C Shim

```
fn main() {
    println!("cargo:rerun-if-changed=src/shim.c");
    println!("cargo:rerun-if-changed=src/shim.h");

    cc::Build::new()
        .file("src/shim.c")
        .warnings(true)
        .compile("shim");
}
```

A missing `rerun-if-changed` can leave stale native code in incremental builds

Linker Directives

```
fn main() {
    println!("cargo:rustc-link-search=native=/opt/vendor/lib");
    println!("cargo:rustc-link-lib=dylib=vendor");
}
```

Linker Directives

```rust
fn main() {
    println!("cargo:rustc-link-search=native=/opt/vendor/lib");
    println!("cargo:rustc-link-lib=dylib=vendor");
}
```

- `static=foo` links `libfoo.a`

Linker Directives

```
fn main() {
    println!("cargo:rustc-link-search=native=/opt/vendor/lib");
    println!("cargo:rustc-link-lib=dylib=vendor");
}
```

- `static=foo` links `libfoo.a`
- `dylib=foo` links a dynamic library

Linker Directives

```
fn main() {
    println!("cargo:rustc-link-search=native=/opt/vendor/lib");
    println!("cargo:rustc-link-lib=dylib=vendor");
}
```

- `static=foo` links `libfoo.a`
- `dylib=foo` links a dynamic library
- Runtime library search is a separate deployment problem

pkg-config

```rust
fn main() {
    pkg_config::Config::new()
        .atleast_version("1.2")
        .probe("libzstd")
        .unwrap();
}
```

pkg-config

```rust
fn main() {
    pkg_config::Config::new()
        .atleast_version("1.2")
        .probe("libzstd")
        .unwrap();
}
```

`pkg-config` asks the host system how to compile and link a native dependency

Generated Bindings

- `bindgen`: C or C++ headers to Rust declarations

Generated Bindings

- `bindgen`: C or C++ headers to Rust declarations
- `cbindgen`: Rust declarations to C headers

Generated Bindings

- `bindgen`: C or C++ headers to Rust declarations
- `cbindgen`: Rust declarations to C headers
- Generated bindings are not automatically safe APIs

Generated Bindings

- `bindgen`: C or C++ headers to Rust declarations
- `cbindgen`: Rust declarations to C headers
- Generated bindings are not automatically safe APIs
- The public wrapper still needs a Rust-level contract

Cross Compilation

- `HOST`: machine running the compiler and build script

Cross Compilation

- `HOST`: machine running the compiler and build script
- `TARGET`: machine the final binary is built for

Cross Compilation

- `HOST`: machine running the compiler and build script
- `TARGET`: machine the final binary is built for
- Rust target support is not enough if native libraries are involved

Cross Compilation

- `HOST`: machine running the compiler and build script
- `TARGET`: machine the final binary is built for
- Rust target support is not enough if native libraries are involved
- You also need the target linker, headers, and libraries

Target Configuration

```
[target.aarch64-unknown-linux-gnu]
linker = "aarch64-linux-gnu-gcc"

[build]
target = "aarch64-unknown-linux-gnu"
```

Target Configuration

```
[target.aarch64-unknown-linux-gnu]
linker = "aarch64-linux-gnu-gcc"

[build]
target = "aarch64-unknown-linux-gnu"
```

The hard part is usually not Rust code generation, but native dependency resolution

File Descriptors and OS Handles

File Descriptors

- Unix file descriptors are small integers indexing kernel state

File Descriptors

- Unix file descriptors are small integers indexing kernel state
- Ownership means responsibility to call `close` exactly once

File Descriptors

- Unix file descriptors are small integers indexing kernel state
- Ownership means responsibility to call `close` exactly once
- Borrowing means temporary use without closing

File Descriptors

- Unix file descriptors are small integers indexing kernel state
- Ownership means responsibility to call `close` exactly once
- Borrowing means temporary use without closing
- Duplicating means creating a second owned descriptor for the same open file description

Raw FD Double Close

```
use std::fs::File;
use std::os::fd::{AsRawFd, FromRawFd};

let file = File::open("data.txt")?;
let fd = file.as_raw_fd();

let also_owner = unsafe { File::from_raw_fd(fd) };
```

Raw FD Double Close

```rust
use std::fs::File;
use std::os::fd::{AsRawFd, FromRawFd};

let file = File::open("data.txt")?;
let fd = file.as_raw_fd();

let also_owner = unsafe { File::from_raw_fd(fd) };
```

Now both `file` and `also_owner` believe they own the same descriptor

Owned Descriptors

```
use std::os::fd::{AsRawFd, FromRawFd, OwnedFd};
use std::io;

fn pipe_owned() -> io::Result<(OwnedFd, OwnedFd)> {
    let mut fds = [-1; 2];
    let rc = unsafe { libc::pipe(fds.as_mut_ptr()) };
    if rc == -1 {
        return Err(io::Error::last_os_error());
    }
```

```
    let read = unsafe { OwnedFd::from_raw_fd(fds[0]) };
    let write = unsafe { OwnedFd::from_raw_fd(fds[1]) };
    Ok((read, write))
}
```

```
    let read = unsafe { OwnedFd::from_raw_fd(fds[0]) };
    let write = unsafe { OwnedFd::from_raw_fd(fds[1]) };
    Ok((read, write))
}
```

The two `from_raw_fd` calls are ownership transfers from the kernel API into Rust

Borrowed Descriptors

```rust
use std::os::fd::{AsRawFd, BorrowedFd};
use std::io;

fn set_nonblocking(fd: BorrowedFd<'_>) -> io::Result<()> {
    let raw = fd.as_raw_fd();
    let flags = unsafe { libc::fcntl(raw, libc::F_GETFL) };
    if flags == -1 {
        return Err(io::Error::last_os_error());
    }
```

```
    let rc = unsafe {
        libc::fcntl(raw, libc::F_SETFL, flags | libc::O_NONBLOCK)
    };
    if rc == -1 {
        return Err(io::Error::last_os_error());
    }
    Ok(())
}
```

```
    let rc = unsafe {
        libc::fcntl(raw, libc::F_SETFL, flags | libc::O_NONBLOCK)
    };
    if rc == -1 {
        return Err(io::Error::last_os_error());
    }
    Ok(())
}
```

`BorrowedFd<'_>` prevents this function from accidentally taking ownership

Syscall Error Shape

- Many C syscalls return `-1` on error and store details in `errno`

Syscall Error Shape

- Many C syscalls return `-1` on error and store details in `errno`
- `EINTR` often means retry

Syscall Error Shape

- Many C syscalls return `-1` on error and store details in `errno`
- `EINTR` often means retry
- `EAGAIN` or `EWOULDBLOCK` means nonblocking operation would block

Syscall Error Shape

- Many C syscalls return `-1` on error and store details in `errno`
- `EINTR` often means retry
- `EAGAIN` or `EWOULDBLOCK` means nonblocking operation would block
- Short reads and short writes are ordinary results, not necessarily errors

Read Loop Skeleton

```rust
fn read_some(fd: BorrowedFd<'_>, buf: &mut [u8]) -> io::Result<usize> {
    loop {
        let n = unsafe {
            libc::read(fd.as_raw_fd(), buf.as_mut_ptr().cast(), buf.len())
        };
        if n >= 0 {
            return Ok(n as usize);
        }
```

```
            let err = io::Error::last_os_error();
            if err.raw_os_error() == Some(libc::EINTR) {
                continue;
            }
            return Err(err);
        }
    }
```

```
            let err = io::Error::last_os_error();
            if err.raw_os_error() == Some(libc::EINTR) {
                continue;
            }
            return Err(err);
        }
    }
```

This is the shape hidden inside many safer I/O abstractions

Unix Domain Socket Path

- `bind` creates a filesystem entry for pathname sockets

Unix Domain Socket Path

- `bind` creates a filesystem entry for pathname sockets
- A stale path from a crashed process can make the next `bind` fail

Unix Domain Socket Path

- `bind` creates a filesystem entry for pathname sockets
- A stale path from a crashed process can make the next `bind` fail
- Filesystem permissions still matter

Unix Domain Socket Path

- `bind` creates a filesystem entry for pathname sockets
- A stale path from a crashed process can make the next `bind` fail
- Filesystem permissions still matter
- Cleanup has to match the socket lifetime, not merely the file descriptor lifetime

Memory Mappings

mmap

- `mmap` creates a virtual memory mapping in the process

mmap

- `mmap` creates a virtual memory mapping in the process
- The result is not Rust allocation and must not be passed to `dealloc`

mmap

- `mmap` creates a virtual memory mapping in the process
- The result is not Rust allocation and must not be passed to `dealloc`
- Failure is `MAP_FAILED`, not a null pointer

mmap

- `mmap` creates a virtual memory mapping in the process
- The result is not Rust allocation and must not be passed to `dealloc`
- Failure is `MAP_FAILED`, not a null pointer
- `munmap` needs the same base pointer and length

Mapping Wrapper I

```
use std::{io, ptr::NonNull};

struct Mapping {
    ptr: NonNull<u8>,
    len: usize,
}

impl Mapping {
    fn anonymous(len: usize) -> io::Result<Self> {
        let raw = unsafe {
            libc::mmap(
```

```
            std::ptr::null_mut(),
            len,
            libc::PROT_READ | libc::PROT_WRITE,
            libc::MAP_PRIVATE | libc::MAP_ANONYMOUS,
            -1,
            0,
        )
    };
```

```
        if raw == libc::MAP_FAILED {
            return Err(io::Error::last_os_error());
        }
        let Some(ptr) = NonNull::new(raw.cast()) else {
            unsafe { libc::munmap(raw, len); }
            return Err(io::Error::other("mmap returned null"));
        };
        Ok(Self { ptr, len })
    }
}
```

Mapping Wrapper II

```
impl Mapping {
    fn as_mut_slice(&mut self) -> &mut [u8] {
        unsafe {
            std::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len)
        }
    }
}

impl Drop for Mapping {
    fn drop(&mut self) {
        unsafe {
            libc::munmap(self.ptr.as_ptr().cast(), self.len);
        }
    }
}
```

Mapping Invariants

- `ptr` is the exact base address returned by `mmap`

Mapping Invariants

- `ptr` is the exact base address returned by `mmap`
- `len` is the exact length used for the mapping

Mapping Invariants

- `ptr` is the exact base address returned by `mmap`
- `len` is the exact length used for the mapping
- `len > 0` for this wrapper

Mapping Invariants

- `ptr` is the exact base address returned by `mmap`
- `len` is the exact length used for the mapping
- `len > 0` for this wrapper
- No Rust reference outlives the mapping

Mapping Invariants

- `ptr` is the exact base address returned by `mmap`
- `len` is the exact length used for the mapping
- `len > 0` for this wrapper
- No Rust reference outlives the mapping
- Mutable slices are not handed out concurrently with aliases

Memory-Mapped Files

- File length and mapping length are separate facts

Memory-Mapped Files

- File length and mapping length are separate facts
- Truncating a mapped file can make later access fault

Memory-Mapped Files

- File length and mapping length are separate facts
- Truncating a mapped file can make later access fault
- Writes through mappings still obey OS synchronization rules

Memory-Mapped Files

- File length and mapping length are separate facts
- Truncating a mapped file can make later access fault
- Writes through mappings still obey OS synchronization rules
- Rust slices cannot express every external invalidation event

Address-Sensitive Values

ManuallyDrop

- `ManuallyDrop<T>` suppresses automatic destructor execution

ManuallyDrop

- `ManuallyDrop<T>` suppresses automatic destructor execution
- It does not make invalid values valid

ManuallyDrop

- `ManuallyDrop<T>` suppresses automatic destructor execution
- It does not make invalid values valid
- It does not erase the field from the type

ManuallyDrop

- `ManuallyDrop<T>` suppresses automatic destructor execution
- It does not make invalid values valid
- It does not erase the field from the type
- After manual drop, the program must track that the value is gone

Manual Drop Example

```
use std::mem::ManuallyDrop;

struct Slot {
    value: ManuallyDrop<String>,
    initialized: bool,
}

impl Slot {
    unsafe fn clear(&mut self) {
        ManuallyDrop::drop(&mut self.value);
        self.initialized = false;
    }
}
```

Manual Drop Example

```
use std::mem::ManuallyDrop;

struct Slot {
    value: ManuallyDrop<String>,
    initialized: bool,
}

impl Slot {
    unsafe fn clear(&mut self) {
        ManuallyDrop::drop(&mut self.value);
        self.initialized = false;
    }
}
```

The boolean is now part of the safety invariant

Self-Reference Is Not Stable

```
use std::ptr;

struct SelfRef {
    data: String,
    ptr: *const String,
}

impl SelfRef {
    fn new(data: String) -> Self {
        let mut out = Self { data, ptr: ptr::null() };
        out.ptr = &out.data;
        out
    }
}
```

Self-Reference Is Not Stable

```
use std::ptr;

struct SelfRef {
    data: String,
    ptr: *const String,
}

impl SelfRef {
    fn new(data: String) -> Self {
        let mut out = Self { data, ptr: ptr::null() };
        out.ptr = &out.data;
        out
    }
}
```

Returning `out` may move it, invalidating the stored address

Pinning the Allocation I

```
use std::marker::PhantomPinned;
use std::pin::Pin;
use std::ptr;

struct StableSelfRef {
    data: String,
    ptr: *const String,
    _pin: PhantomPinned,
}
```

```
impl StableSelfRef {
    fn new(data: String) -> Pin<Box<Self>> {
        let mut boxed = Box::pin(Self {
            data,
            ptr: ptr::null(),
            _pin: PhantomPinned,
        });
```

```
        let ptr = &boxed.data as *const String;
        unsafe {
            Pin::as_mut(&mut boxed).get_unchecked_mut().ptr = ptr;
        }
        boxed
    }
}
```

```
        let ptr = &boxed.data as *const String;
        unsafe {
            Pin::as_mut(&mut boxed).get_unchecked_mut().ptr = ptr;
        }
        boxed
    }
}
```

The unsafe assignment relies on the boxed allocation not moving afterwards

Pin Contract

- `Pin<P>` is a promise about the pointee, not about the pointer object

Pin Contract

- `Pin<P>` is a promise about the pointee, not about the pointer object
- `PhantomPinned` prevents automatic `Unpin`

Pin Contract

- `Pin<P>` is a promise about the pointee, not about the pointer object
- `PhantomPinned` prevents automatic `Unpin`
- Moving `Pin<Box<T>>` is fine; moving the boxed `T` is not

Pin Contract

- `Pin<P>` is a promise about the pointee, not about the pointer object
- `PhantomPinned` prevents automatic `Unpin`
- Moving `Pin<Box<T>>` is fine; moving the boxed `T` is not
- Field projection from pinned structs is where many bugs enter

Target-Specific Code

Conditional Compilation

```rust
#[cfg(target_family = "unix")]
fn platform_name() -> &'static str {
    "unix"
}

#[cfg(target_family = "windows")]
fn platform_name() -> &'static str {
    "windows"
}
```

Conditional Compilation

```rust
#[cfg(target_family = "unix")]
fn platform_name() -> &'static str {
    "unix"
}

#[cfg(target_family = "windows")]
fn platform_name() -> &'static str {
    "windows"
}
```

`cfg` removes code before type checking for the other platform

CPU Features

```rust
#[cfg(target_arch = "x86_64")]
fn has_avx2() -> bool {
    std::is_x86_feature_detected!("avx2")
}
```

CPU Features

```rust
#[cfg(target_arch = "x86_64")]
fn has_avx2() -> bool {
    std::is_x86_feature_detected!("avx2")
}
```

- Compiling for an architecture is not the same as enabling every CPU feature

CPU Features

```rust
#[cfg(target_arch = "x86_64")]
fn has_avx2() -> bool {
    std::is_x86_feature_detected!("avx2")
}
```

- Compiling for an architecture is not the same as enabling every CPU feature
- Dispatch can be runtime or compile-time

CPU Features

```rust
#[cfg(target_arch = "x86_64")]
fn has_avx2() -> bool {
    std::is_x86_feature_detected!("avx2")
}
```

- Compiling for an architecture is not the same as enabling every CPU feature
- Dispatch can be runtime or compile-time
- Calling feature-specific code without the feature is undefined behavior

Target Features

```rust
#[cfg(target_arch = "x86_64")]
#[target_feature(enable = "avx2")]
unsafe fn avx2_impl(input: &[u8]) -> usize {
    input.len()
}
```

Target Features

```rust
#[cfg(target_arch = "x86_64")]
#[target_feature(enable = "avx2")]
unsafe fn avx2_impl(input: &[u8]) -> usize {
    input.len()
}
```

A safe caller must check the feature before calling this function

Inline Assembly

```
#[cfg(target_arch = "x86_64")]
fn read_tsc() -> u64 {
    let hi: u32;
    let lo: u32;
    unsafe {
        std::arch::asm!(
            "rdtsc",
```

```
            out("edx") hi,
            out("eax") lo,
            options(nomem, nostack, preserves_flags),
        );
    }
    ((hi as u64) << 32) | lo as u64
}
```

```
            out("edx") hi,
            out("eax") lo,
            options(nomem, nostack, preserves_flags),
        );
    }
    ((hi as u64) << 32) | lo as u64
}
```

The operand list and options are part of the correctness argument

Assembly Contracts

- Registers must match the instruction and ABI

Assembly Contracts

- Registers must match the instruction and ABI
- `nomem` is only legal if the assembly does not read or write memory

Assembly Contracts

- Registers must match the instruction and ABI
- `nomem` is only legal if the assembly does not read or write memory
- `preserves_flags` is only legal if flags are preserved

Assembly Contracts

- Registers must match the instruction and ABI
- `nomem` is only legal if the assembly does not read or write memory
- `preserves_flags` is only legal if flags are preserved
- Target-specific code must be hidden behind `cfg` or feature checks

Questions?